# Predicting Housing Prices Using Neighborhood-Level Data in Charlotte

Pavle Dimitrijevic, Ikumi Uemura, Michael Forshay, Sonia Sun, and James Harris

## Introduction

This project examines how neighborhood-level economic, educational, and public safety factors influence housing prices in Charlotte, North Carolina. Using data aggregated at the Neighborhood Profile Area (NPA) level, we build both regression and classification models to better understand and predict variation in housing values across neighborhoods. Housing prices were selected as the primary outcome because they reflect broader economic conditions and quality of life differences. Being based in Charlotte, we chose this topic because housing affordability is an active and consequential issue in one of the fastest-growing cities in the Southeast, and we wanted to understand what neighborhood-level data could tell us about the forces driving price variation.

The analysis uses a merged dataset combining multiple sources, including the Charlotte-Mecklenburg Quality of Life dataset and City of Charlotte Open Data. Key variables include employment rates, graduation rates, job density, prior housing prices, and homicide counts. These features were selected to capture both economic strength and neighborhood conditions.

Across the models tested, prior housing prices (home_price_2021) emerged as the strongest predictor of current housing prices. For the regression task, linear regression provided the best overall performance, achieving the lowest RMSE among the models considered. For the classification task, a k-nearest neighbors model (k=5) performed best based on ROC-AUC, indicating moderate ability to distinguish between higher- and lower-value neighborhoods.

The remainder of this report details the modeling process and results. First, we present the regression model and evaluate its performance and key predictors. Next, we present the classification model and its ability to categorize neighborhoods. The report concludes with a summary of findings and their implications for understanding housing price variation.

## Regression Model

The goal of the regression model is to predict 2023 housing prices across Charlotte neighborhoods using economic, educational, and public safety variables. The target variable is home_price_2023, and the features used include employment_2023, grad_2023, job_density_2022, homicide_count, and home_price_2021. The inclusion of home_price_2021 serves as a key engineered feature, capturing prior housing trends.

We began with linear regression as a baseline given the continuous nature of the target variable. KNN and random forest were then evaluated as alternatives to capture potential non-linear relationships in the data. Hyperparameters for kNN were selected using cross-validation, specifically, a 5-fold GridSearchCV across $k \in \{3, 5, 7, 9, 11, 13, 15\}$ found that k=11 minimized cross-validated RMSE on the training set. Model performance was assessed using Root Mean Squared Error (RMSE) and $R^2$.

Among the models tested, linear regression performed best, achieving the lowest RMSE of approximately 280,805 and an $R^2$ of approximately 0.395. While random forest and KNN produced competitive cross-validated RMSEs (~253,475 and ~254,476 respectively) compared to linear regression (~228,787), linear regression achieved the lowest held-out test RMSE of 280,805, indicating it generalizes better to unseen data for this particular dataset. An $R^2$ of 0.395 means the model explains approximately 40% of the variation in 2023 housing prices, sufficient for identifying broad neighborhood trends but not precise enough for individual property valuation. In comparison, kNN regression produced a higher RMSE (~327,007) and lower explanatory power ($R^2 \approx 0.18$), while the random forest model performed moderately (RMSE ≈ 300,969, $R^2 \approx 0.31$). These results indicate that a relatively simple linear relationship captures the structure in the data better than more complex models for this task.

Furthermore, visual diagnostics support this conclusion. The Actual vs. Predicted plot shows that the model performs reasonably well for mid-range housing prices but tends to underpredict higher-priced homes, as indicated by points falling below the ideal line. This pattern is further reinforced in the residual plot, where residuals increase in magnitude for higher predicted values, suggesting non-constant variance and reduced accuracy at the upper end of the market.

Feature importance analysis from the random forest model highlights that home_price_2021 is by far the most influential predictor, followed by employment and graduation rates. Job density and homicide count contribute less to predictive performance. This indicates that past housing prices and economic conditions play a much larger role in determining current housing values than public safety measures in this dataset.

Overall, the regression analysis suggests that housing prices in Charlotte are strongly driven by prior market conditions and, to a lesser extent, by economic and educational factors. While the linear model provides the best performance among those tested, its limitations in predicting high-value homes highlight areas where additional features or more complex modeling approaches could improve accuracy.

## Classification Model

The goal of the classification model is to categorize neighborhoods as high-value or low-value housing areas. The target variable, high_price_2023, is a binary indicator where neighborhoods with housing prices above the median are classified as 1 and those below the median are classified as 0. The feature set includes employment_2023, grad_2023, job_density_2022, homicide_count, and home_price_2021, consistent with the regression model.

The dataset is well-balanced, with approximately equal proportions of high- and low-value neighborhoods, which ensures that model performance is not biased toward one class. We evaluated multiple classification approaches, including logistic regression, k-nearest neighbors (kNN), and random forest classification. The optimal value for kNN was selected using cross-validation, specifically, a 5-fold GridSearchCV across $k \in \{3, 5, 7, 9, 11, 13, 15\}$ using ROC-AUC as the scoring criteria found k=5 to be optimal on the training set. Model performance was assessed using accuracy, ROC-AUC, and confusion matrices, with primary emphasis on ROC-AUC.

Among the models tested, k-nearest neighbors (k=5) achieved the highest ROC-AUC of approximately 0.685, indicating the strongest ability to distinguish between high- and low-value neighborhoods. Logistic regression performed slightly worse (ROC-AUC ≈ 0.651), while the random forest model produced a moderate ROC-AUC of approximately 0.662. In terms of accuracy, kNN also performed best (~0.659), though differences across models were relatively small.

Cross-validation results provide additional context for model stability. Logistic regression and random forest models achieved similar average ROC-AUC scores (~0.68), while kNN showed lower cross-validated performance (~0.60), suggesting that although kNN performs well on the test set, it may be more sensitive to variation in the data. Despite this instability, KNN was selected as the final model because it achieved the highest test ROC-AUC (0.685) and the best recall for high-value neighborhoods (71%), making it the most practically useful model for identifying premium areas. A ROC-AUC of 0.685 means the model correctly ranks a randomly chosen high-value neighborhood above a randomly chosen low-value neighborhood approximately 68.5% of the time, meaningfully above random chance but not strong enough to serve as a standalone decision tool.

The confusion matrices indicate that all models make a relatively balanced mix of classification errors, with no extreme bias toward predicting either class. This aligns with the balanced nature of the dataset and supports the validity of the evaluation metrics.

Feature importance analysis from the random forest model again shows that home_price_2021 is the most influential predictor, followed by employment and graduation rates. Job density and

homicide count contribute less to classification performance, reinforcing the idea that prior housing prices and economic factors are the primary drivers of neighborhood classification.

Overall, the classification model demonstrates a moderate ability to distinguish between higher- and lower-value neighborhoods. While kNN provides the best performance based on ROC-AUC, the results suggest that classification in this context is inherently more challenging than regression, likely due to overlapping characteristics between neighborhoods.

## Conclusion

This project examined how neighborhood-level economic, educational, and public safety factors influence housing prices in Charlotte using both regression and classification models. By merging multiple data sources at the NPA level, we developed a consistent dataset that allowed for both prediction and categorization of housing values.

The results show that prior housing prices (home_price_2021) are the strongest predictor of current housing prices, highlighting the persistence of housing market trends over time. Economic factors such as employment and graduation rates also contribute meaningfully, while job density and homicide count have more limited impact in this dataset.

For the regression task, linear regression provided the best performance, indicating that the relationship between predictors and housing prices is largely linear within the range of the data. For the classification task, k-nearest neighbors achieved the highest ROC-AUC, demonstrating moderate ability to distinguish between higher- and lower-value neighborhoods, though results varied across validation methods.

Overall, the findings suggest that housing price differences across neighborhoods are driven primarily by historical price patterns and economic conditions rather than public safety measures alone. While the models provide useful predictive insight, their limitations, particularly in predicting higher-priced homes and in classification accuracy, indicate that additional features or more complex modeling approaches may be needed to improve performance.

These results provide a data-driven perspective on housing value variation across Charlotte neighborhoods. The dominance of prior prices suggests that housing inequality in Charlotte is largely self-reinforcing, neighborhoods that were expensive in 2021 remain expensive in 2023 regardless of current economic or safety conditions. For policymakers and community organizations, this implies that improving employment or educational outcomes alone may be insufficient to shift a neighborhood's price trajectory and structural interventions targeting historical pricing disparities may be necessary to meaningfully address housing inequality across the city.

*Appendix:*

*Regression Model Code*

```
# RegressionModel.py

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import mean_squared_error, r2_score

from sklearn.linear_model import LinearRegression

from sklearn.neighbors import KNeighborsRegressor

from sklearn.ensemble import RandomForestRegressor

# -------------------------

# 1. Load Data

# -------------------------

df = pd.read_csv("final_fixed_dataset.csv")

df = df.dropna()

# -------------------------

# 2. Feature Selection

# -------------------------

features = [

   "employment_2023",

   "grad_2023",

   "job_density_2022",

   "homicide_count",

   "home_price_2021"
```

```
]

X = df[features]

y = df["home_price_2023"]

# -------------------------

# 3. Train/Test Split

# -------------------------

X_train, X_test, y_train, y_test = train_test_split(

   X, y, test_size=0.2, random_state=42

)

# -------------------------

# 4. Scaling (for Linear + KNN)

# -------------------------

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

# -------------------------

# 5. KNN Tuning (CV)

# -------------------------

param_grid = {

   "n_neighbors": list(range(3, 16, 2))

}

knn_grid = GridSearchCV(

   KNeighborsRegressor(),

   param_grid,

   cv=5,

   scoring='neg_mean_squared_error'
```

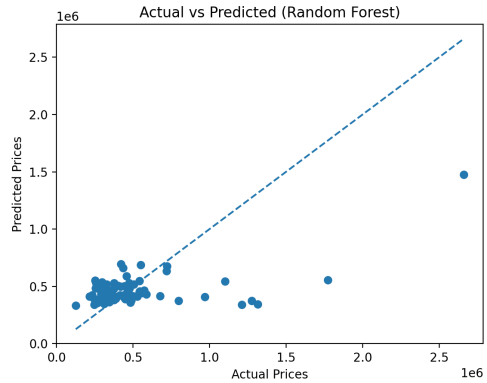
```
)

knn_grid.fit(X_train_scaled, y_train)

best_k = knn_grid.best_params_["n_neighbors"]

print(f"\nBest K for KNN (CV): {best_k}")

# -------------------------

# 6. Define Models

# -------------------------

models = {

   "Linear Regression": LinearRegression(),

   f"KNN (k={best_k})": KNeighborsRegressor(n_neighbors=best_k),

   "Random Forest": RandomForestRegressor(random_state=42)

}

# 7. Train Models

trained_models = {}

trained_models["Linear Regression"] = models["Linear Regression"].fit(X_train_scaled, y_train)

trained_models[f"KNN (k={best_k})"] = models[f"KNN (k={best_k})"].fit(X_train_scaled, y_train)

trained_models["Random Forest"] = models["Random Forest"].fit(X_train, y_train)

# -------------------------

# 8. Evaluation Function

# -------------------------

results = []

def evaluate_model(name, model, X_te):

   y_pred = model.predict(X_te)

   rmse = float(np.sqrt(mean_squared_error(y_test, y_pred)))

   r2 = float(r2_score(y_test, y_pred))

   results.append((name, rmse, r2))
```

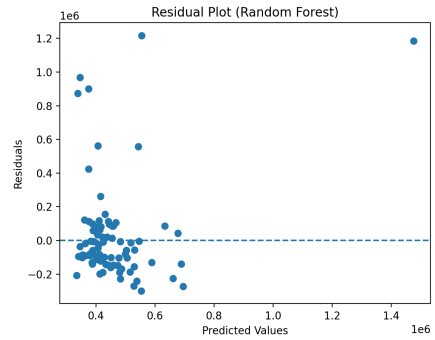
```
    print(f"\n{name}")

    print("-" * 40)

    print("RMSE:", round(rmse, 4))

    print("R²:", round(r2, 4))

# Run evaluations

evaluate_model("Linear Regression", trained_models["Linear Regression"], X_test_scaled)

evaluate_model(f"KNN (k={best_k})", trained_models[f"KNN (k={best_k})"], X_test_scaled)

evaluate_model("Random Forest", trained_models["Random Forest"], X_test)

# -------------------------

# 9. Model Comparison

# -------------------------

print("\nModel Comparison (RMSE, R²):")

for r in results:

    print(r)

best = min(results, key=lambda x: x[1])  # lowest RMSE

print(f"\nBest Model: {best[0]} with RMSE = {round(best[1], 4)}")

# -------------------------

# 10. Cross Validation

# -------------------------

print("\nCross-Validation (RMSE):")

for name, model in models.items():

    if "Random Forest" in name:

        X_use = X

    else:

        X_use = scaler.fit_transform(X)

    scores = cross_val_score(model, X_use, y, cv=5, scoring='neg_mean_squared_error')
```

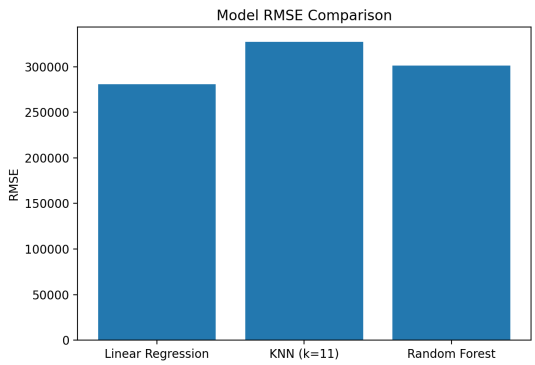
```
    rmse_scores = np.sqrt(-scores)

    print(f"{name}: Mean RMSE = {round(rmse_scores.mean(), 4)}"

# -------------------------

# 11. Feature Importance

# -------------------------

rf = trained_models["Random Forest"]

feature_importance = pd.DataFrame({

    "Feature": features,

    "Importance": rf.feature_importances_

}).sort_values(by="Importance", ascending=False)

print("\nFeature Importances (Random Forest):")

print(feature_importance)

# -------------------------

# 12. Actual vs Predicted

# -------------------------

plt.figure()

y_pred_rf = trained_models["Random Forest"].predict(X_test)

plt.scatter(y_test, y_pred_rf)

plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], linestyle='--')

plt.xlabel("Actual Prices")

plt.ylabel("Predicted Prices")

plt.title("Actual vs Predicted (Random Forest)")

plt.show()

# -------------------------

# 13. Residual Plot

# -------------------------
```

```
residuals = y_test - y_pred_rf

plt.figure()

plt.scatter(y_pred_rf, residuals)

plt.axhline(0, linestyle='--')

plt.xlabel("Predicted Values")

plt.ylabel("Residuals")

plt.title("Residual Plot (Random Forest)")

plt.show()

# -------------------------

# 14. Feature Importance Plot

# -------------------------

plt.figure()

plt.barh(feature_importance["Feature"], feature_importance["Importance"])

plt.gca().invert_yaxis()

plt.xlabel("Importance")

plt.title("Feature Importance (Random Forest)")

plt.show()

# -------------------------

# 15. RMSE Comparison Plot

names = [r[0] for r in results]

rmse_vals = [r[1] for r in results]

plt.figure()

plt.bar(names, rmse_vals)

plt.ylabel("RMSE")

plt.title("Model RMSE Comparison")

plt.show()
```

# *Regression Model Output*

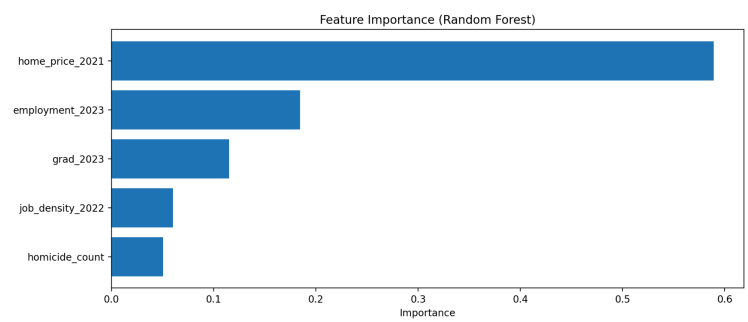



```
Model Comparison (RMSE, R²):
('Linear Regression', 280805.0248044438, 0.3953918812344722)
('KNN (k=11)', 327006.6420915522, 0.18006887470567778)
('Random Forest', 300969.04139770876, 0.30544302844131876)

Best Model: Linear Regression with RMSE = 280805.0248

Cross-Validation (RMSE):
Linear Regression: Mean RMSE = 228787.4479
KNN (k=11): Mean RMSE = 254475.5079
Random Forest: Mean RMSE = 253474.5777

Feature Importances (Random Forest):
            Feature  Importance
4   home_price_2021    0.589297
0   employment_2023    0.184501
1         grad_2023    0.115091
2  job_density_2022    0.060361
3    homicide_count    0.050750
```

```
Best K for KNN (CV): 11

Linear Regression
----------------------------------------
RMSE: 280805.0248
R²: 0.3954

KNN (k=11)
----------------------------------------
RMSE: 327006.6421
R²: 0.1801

Random Forest
----------------------------------------
RMSE: 300969.0414
R²: 0.3054
```



*Classification Model Code*

```
# ClassificationModel.py

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, cross_val_score

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import accuracy_score, roc_auc_score, confusion_matrix,
classification_report, roc_curve

from sklearn.linear_model import LogisticRegression

from sklearn.neighbors import KNeighborsClassifier

from sklearn.ensemble import RandomForestClassifier

# -------------------------

# 1. Load Data

# -------------------------

df = pd.read_csv("final_fixed_dataset.csv")

df = df.dropna()

# -------------------------

# 2. Create Target Variable

# -------------------------

df["high_price_2023"] = (df["home_price_2023"] > df["home_price_2023"].median()).astype(int)

# -------------------------

# 3. Feature Selection

# -------------------------

features = [

   "employment_2023",

   "grad_2023",
```

```
    "job_density_2022",
    "homicide_count",
    "home_price_2021"
]
X = df[features]
y = df["high_price_2023"]
# -------------------------
# 4. Class Balance
# -------------------------
print("\nClass Distribution:")
print(y.value_counts())
print("\nClass Proportions:")
print(y.value_counts(normalize=True))
# -------------------------
# 5. Train/Test Split
# -------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
# -------------------------
# 6. Scaling
# -------------------------
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# -------------------------
```

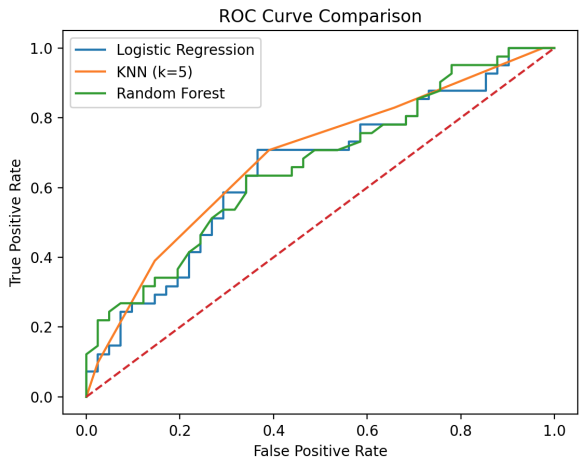
```
# 7. KNN Tuning (Proper CV)

# -------------------------

from sklearn.model_selection import GridSearchCV

param_grid = {

   "n_neighbors": list(range(3, 16, 2))

}

knn_grid = GridSearchCV(

   KNeighborsClassifier(),

   param_grid,

   cv=5,

   scoring='roc_auc'

)

knn_grid.fit(X_train_scaled, y_train)

best_k = knn_grid.best_params_["n_neighbors"]

print(f"\nBest K for KNN (CV): {best_k}")

# -------------------------

# 8. Define Models

# -------------------------

models = {

   "Logistic Regression": LogisticRegression(max_iter=1000),

   f"KNN (k={best_k})": KNeighborsClassifier(n_neighbors=best_k),

   "Random Forest": RandomForestClassifier(random_state=42)

}

# -------------------------

# 9. Train Models

# -------------------------
```

```
trained_models = {}

trained_models["Logistic Regression"] = models["Logistic Regression"].fit(X_train_scaled,
y_train)

trained_models[f"KNN (k={best_k})"] = models[f"KNN (k={best_k})"].fit(X_train_scaled, y_train)

trained_models["Random Forest"] = models["Random Forest"].fit(X_train, y_train)

# -------------------------

# 10. Evaluation

# -------------------------

results = []

def evaluate_model(name, model, X_te):

    y_pred = model.predict(X_te)

    y_prob = model.predict_proba(X_te)[:, 1]

    acc = float(accuracy_score(y_test, y_pred))

    auc = float(roc_auc_score(y_test, y_prob))

    results.append((name, acc, auc))

    print(f"\n{name}")

    print("-" * 40)

    print("Accuracy:", round(acc, 4))

    print("ROC-AUC:", round(auc, 4))

    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

    print("Classification Report:\n", classification_report(y_test, y_pred))

# Run evaluations

evaluate_model("Logistic Regression", trained_models["Logistic Regression"], X_test_scaled)

evaluate_model(f"KNN (k={best_k})", trained_models[f"KNN (k={best_k})"], X_test_scaled)

evaluate_model("Random Forest", trained_models["Random Forest"], X_test)

# -------------------------

# 11. Model Comparison
```

```python
# -------------------------
print("\nModel Comparison (Accuracy, ROC-AUC):")
for r in results:
    print(r)
best = max(results, key=lambda x: x[2])
print(f"\nBest Model: {best[0]} with ROC-AUC = {round(best[2], 4)}")
# -------------------------
# 12. Cross Validation
# -------------------------
print("\nCross-Validation (ROC-AUC):")
for name, model in models.items():
    if "Random Forest" in name:
        X_use = X
    else:
        X_use = scaler.fit_transform(X)
    scores = cross_val_score(model, X_use, y, cv=5, scoring='roc_auc')
    print(f"{name}: Mean ROC-AUC = {round(scores.mean(), 4)}")
# -------------------------
# 13. Feature Importance
# -------------------------
rf = trained_models["Random Forest"]
feature_importance = pd.DataFrame({
    "Feature": features,
    "Importance": rf.feature_importances_
}).sort_values(by="Importance", ascending=False)
print("\nFeature Importances (Random Forest):")
```

```python
print(feature_importance)

# -------------------------

# 14. ROC Curve

# -------------------------

plt.figure()

for name, model in trained_models.items():

   if "Random Forest" in name:

       X_te = X_test

   else:

       X_te = X_test_scaled

   y_prob = model.predict_proba(X_te)[:, 1]

   fpr, tpr, _ = roc_curve(y_test, y_prob)

   plt.plot(fpr, tpr, label=name)

plt.plot([0, 1], [0, 1], linestyle='--')

plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

plt.title("ROC Curve Comparison")

plt.legend()

plt.show()
```

# *Classification Model Output*




```
Class Distribution:
high_price_2023
0    205
1    204
Name: count, dtype: int64

Class Proportions:
high_price_2023
0    0.501222
1    0.498778
Name: proportion, dtype: float64

Best K for KNN (CV): 5
```

```
Logistic Regression
--------------------------------------
Accuracy: 0.6463
ROC-AUC: 0.6514
Confusion Matrix:
 [[27 14]
 [15 26]]
Classification Report:
              precision    recall  f1-score   support

           0       0.64      0.66      0.65        41
           1       0.65      0.63      0.64        41

    accuracy                           0.65        82
   macro avg       0.65      0.65      0.65        82
weighted avg       0.65      0.65      0.65        82
```

```
KNN (k=5)
--------------------------------------
Accuracy: 0.6585
ROC-AUC: 0.6853
Confusion Matrix:
 [[25 16]
 [12 29]]
Classification Report:
              precision    recall  f1-score   support

           0       0.68      0.61      0.64        41
           1       0.64      0.71      0.67        41

    accuracy                           0.66        82
   macro avg       0.66      0.66      0.66        82
weighted avg       0.66      0.66      0.66        82
```

```
Random Forest
--------------------------------------
Accuracy: 0.5976
ROC-AUC: 0.6618
Confusion Matrix:
 [[23 18]
 [15 26]]
Classification Report:
              precision    recall  f1-score   support

           0       0.61      0.56      0.58        41
           1       0.59      0.63      0.61        41

    accuracy                           0.60        82
   macro avg       0.60      0.60      0.60        82
weighted avg       0.60      0.60      0.60        82
```

```
Cross-Validation (ROC-AUC):
Logistic Regression: Mean ROC-AUC = 0.6842
KNN (k=5): Mean ROC-AUC = 0.6038
Random Forest: Mean ROC-AUC = 0.6807

Feature Importances (Random Forest):
             Feature  Importance
4   home_price_2021    0.381914
0   employment_2023    0.208131
1         grad_2023    0.197602
2  job_density_2022    0.117732
3    homicide_count    0.094620
```

```
Model Comparison (Accuracy, ROC-AUC):
('Logistic Regression', 0.6463414634146342, 0.6513979773944081)
('KNN (k=5)', 0.6585365853658537, 0.6853063652587745)
('Random Forest', 0.5975609756097561, 0.6618084473527661)

Best Model: KNN (k=5) with ROC-AUC = 0.6853
```

*SQL Merging Evidence*

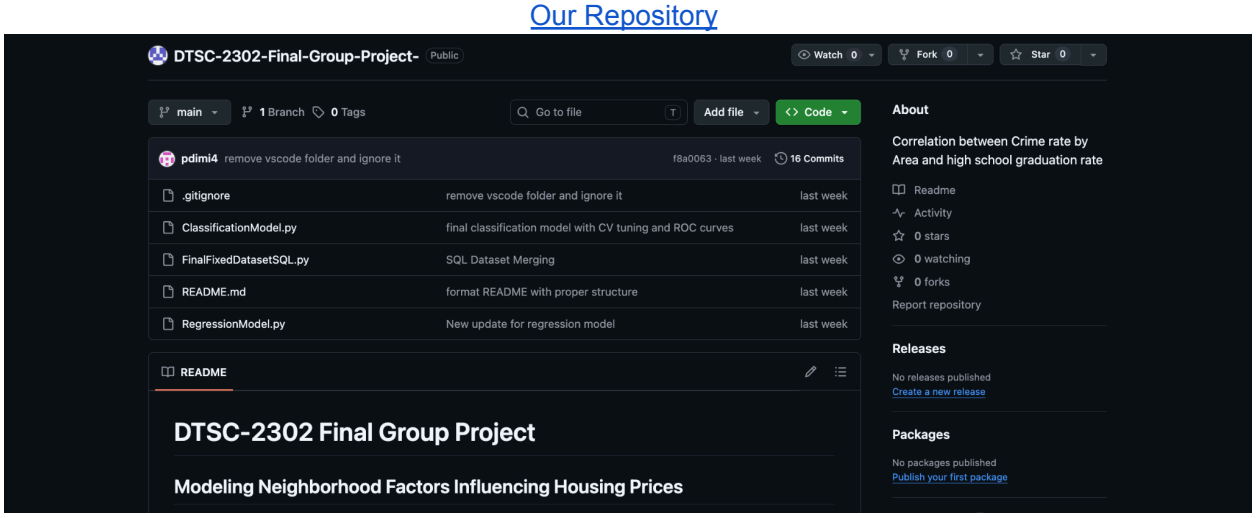
```
# FinalFixedDatasetSQL.py

import pandas as pd

import sqlite3

# -------------------------

# 1. Load datasets (YOUR PATHS)

# -------------------------

employment = pd.read_csv("/Users/pavledimitrijevic/Downloads/Employment.csv")

graduation = pd.read_csv("/Users/pavledimitrijevic/Downloads/High School Graduation Rate.csv")

housing = pd.read_csv("/Users/pavledimitrijevic/Downloads/Home Sales Price.csv")

job_density = pd.read_csv("/Users/pavledimitrijevic/Downloads/Job Density.csv")

crime = pd.read_csv("/Users/pavledimitrijevic/Downloads/CMPD_Homicide.csv")

# -------------------------

# 2. CLEAN DATA (for SQL only)

# -------------------------

employment["2023"] = employment["2023"].replace("--", None)

graduation["2023"] = graduation["2023"].replace("--", None)

employment["2023"] = employment["2023"].str.replace("%", "", regex=False)

graduation["2023"] = graduation["2023"].str.replace("%", "", regex=False)

employment["2023"] = pd.to_numeric(employment["2023"], errors="coerce")

graduation["2023"] = pd.to_numeric(graduation["2023"], errors="coerce")

housing["2021"] = housing["2021"].replace(r"[\$,]", "", regex=True)

housing["2023"] = housing["2023"].replace(r"[\$,]", "", regex=True)

housing["2021"] = pd.to_numeric(housing["2021"], errors="coerce")

housing["2023"] = pd.to_numeric(housing["2023"], errors="coerce")

# -------------------------
```

```
# 3. SQL database

# -------------------------

conn = sqlite3.connect(":memory:")

employment.to_sql("Employment", conn, index=False, if_exists="replace")

graduation.to_sql("Graduation", conn, index=False, if_exists="replace")

housing.to_sql("Housing", conn, index=False, if_exists="replace")

job_density.to_sql("JobDensity", conn, index=False, if_exists="replace")

crime.to_sql("Crime", conn, index=False, if_exists="replace")

# -------------------------

# 4. SQL Query

# -------------------------

query = """

WITH crime_agg AS (

   SELECT

       NPA,

       COUNT(*) AS homicide_count

   FROM Crime

   GROUP BY NPA

)

SELECT

   e.NPA,

   e."2023" AS employment_2023,

   g."2023" AS grad_2023,

   j."2022" AS job_density_2022,

   h."2021" AS home_price_2021,

   h."2023" AS home_price_2023,
```

```
   COALESCE(c.homicide_count, 0) AS homicide_count

FROM Employment e

INNER JOIN Graduation g ON e.NPA = g.NPA

INNER JOIN JobDensity j ON e.NPA = j.NPA

INNER JOIN Housing h ON e.NPA = h.NPA

LEFT JOIN crime_agg c ON e.NPA = c.NPA

ORDER BY e.NPA

"""

# 5. Execute SQL

final_dataset = pd.read_sql(query, conn)

# 6. FORMAT

# -------------------------

# Add % back

final_dataset["employment_2023"] = final_dataset["employment_2023"].round(1).astype(str) + "%"

final_dataset["grad_2023"] = final_dataset["grad_2023"].round(1).astype(str) + "%"

# Add $ + commas back

final_dataset["home_price_2021"] = final_dataset["home_price_2021"].apply(lambda x:
f"${x:,.0f}")

final_dataset["home_price_2023"] = final_dataset["home_price_2023"].apply(lambda x:
f"${x:,.0f}")

# Ensure homicide_count is int

final_dataset["homicide_count"] = final_dataset["homicide_count"].astype(int)

# 7. Output

print("\nFinal Dataset Preview:")

print(final_dataset.head())

print("\nData Types:")

print(final_dataset.dtypes)
```

## *SQL Merging Output*

```
All files imported into database.
SQL merge completed.

Preview:
   NPA  employment_2023  grad_2023  home_price_2023  home_price_2021  \
0    2             95.6      100.0         488364.0         336581.0
1    3             97.4      100.0         667092.0         561120.0
2    4             94.3      100.0        1493043.0        1295555.0
3    5             82.8       83.3         255031.0         153707.0
4    6            100.0       90.5         445698.0         225545.0

   job_density_2022  homicide_count
0               1.0               2
1              31.0               5
2               0.0               0
3               1.0               3
4               2.0              11

Shape:
(459, 7)
Created: housing_project.db
Created: final_merged_dataset.csv
```

## *Github Repository*

[Our Repository]